

Building a Pascal Compiler

Stage 1: a shared front end and an optimising JVM/Jasmin back end.

Stage 2: your own bytecode for an adaptive virtual machine.

REPOSITORY	netos23/Pascal2J — Kotlin 2.3.x, Gradle KTS, JVM toolchain 18
STATUS	Grammar drafted; implementation not yet started (old Java prototype archived)
TARGET 1	Jasmin assembly → <code>.class</code> , semantically faithful, then optimised
TARGET 2	Own instruction set + profiling, tiered, deoptimising VM
SOURCES	Wirth, <i>Compiler Construction</i> (1996) · Aho, Lam, Sethi & Ullman, <i>Compilers</i> (2nd ed., “Dragon”) · Appel, <i>Modern Compiler Implementation in Java</i> (2nd ed.) · Pierce, <i>Types and Programming Languages</i>

This document is a **roadmap, not a solution**. It fixes the order of work, names the decisions you must make yourself, points at the chapter that answers each question, and lists the exercises that are worth your time. Where it gives an opinion it says so, and says why.

Contents

Fourteen parts. Parts 0–3 are decisions and scaffolding; 4–11 are the compiler; 12–13 look ahead to the VM.

0 How to use this roadmap

The four books, and what each is for

1 Scope: which Pascal are you writing?

Dialect, feature triage, the deviations file

2 Architecture and the front/back boundary

Modules, the two IR levels, three executors

3 Milestone map

M0–M12, exit criteria, the vertical slice

4 Foundations: sources, spans, diagnostics

5 Lexical analysis

6 Parsing and the AST

7 Scopes, symbols and nesting

8 Types and semantic analysis

Writing the rules down before coding them

9 HIR: desugaring and lowering

10 MIR: CFG, SSA, and the optimiser

11 The Jasmin back end

Mapping, verification, runtime library

12 Testing, conformance, measurement

13 Stage 2: your own bytecode and adaptive VM

A Decision log template

B Suggested repository layout

C Cross-reference matrix

D Beyond the four books

E Risk register and anti-patterns

How to use this roadmap

You have four books that disagree with each other in useful ways. Read them as four opinions on the same problem, not as four chapters of one manual.

Every part below has the same shape: **goal**, **deliverable**, the **decisions** that are genuinely yours to make, a **reading** strip with chapter pointers, **traps** that cost people weeks, and **tasks** — small, self-contained pieces of work that teach something you cannot learn by reading. Tasks are marked (1) an evening, (2) a weekend, (3) a week or more.

Nothing here is implemented for you on purpose. A compiler is one of the few programs where the design decisions *are* the education; if you take a decision from a document instead of making it, you will hit the consequence three months later without the context to understand it.

The four books, and what each is for

BOOK	ROLE	HOW TO ACTUALLY USE IT
Wirth Compiler Construction, 1996	The skeleton	A complete compiler in ~130 pages. Its value is <i>proportion</i> : it shows how small each phase can be when the language is designed for one pass. Read chapters 1–12 once, in order, early. Wirth's Oberon-0 compiler emits code directly from the parser with no IR at all — you will not do that, but you should understand why he can and why you cannot (two back ends, an optimiser). Chapters 9–12 (RISC target, expressions, control flow, procedures and locality) are the clearest short account of code generation anywhere; chapter 12 on the static link is exactly the problem the JVM will make hard for you.
Dragon Aho, Lam, Sethi, Ullman, 2nd ed.	The encyclopedia	Do not read cover to cover; you will stall in chapter 4 on LR construction you will never write by hand. Use it as a lookup: §3.3–3.7 for regular languages and DFA construction, §4.4 for LL(1) and FIRST/FOLLOW, ch. 5 for syntax-directed definitions (the right mental model for your semantic passes), §6.2 for three-address code and §6.2.4 for SSA, §6.6–6.7 for boolean-expression translation and backpatching, §7.3 for access to non-local data, ch. 8 for basic blocks, peephole and §8.10 Sethi–Ullman, ch. 9 for data-flow frameworks and loop optimisation.
Appel Modern Compiler Implementation in Java, 2nd ed.	The blueprint	Closest to what you are building: a real multi-phase compiler with clean module boundaries, in a JVM language. Chapters 4–8 (abstract syntax, semantic analysis, activation records, translation to IR trees, basic blocks and traces) are your Parts 6–10. The <i>Ex / Nx / Cx</i> trichotomy in ch. 7 is the single most useful idea in the book for your back end. Chapters 10–11 (liveness, register allocation) you can skip for the JVM and return to for stage 2. Chapter 13 (garbage collection) and 15 (closures) become essential in stage 2. Chapters 17–19 (data-flow, loops, SSA) are your optimiser.
Pierce Types and Programming Languages	The rigour	Pascal is simply typed, so you need a narrow slice: ch. 3 (induction and operational semantics — the notation you will use to <i>write your own rules down</i>), ch. 8–9 (typed arithmetic, STLC, and what progress and preservation mean), ch. 11 (records, variants, sums, general recursion), ch. 13 (references) — this is your <code>^T</code> pointers and <code>var</code> parameters, ch. 14 (exceptions) for runtime errors, ch. 15–17 (subtyping) for the subrange/assignment-compatibility relation, and ch. 19 (Featherweight Java) when you decide how records and procedures map onto classes. Use it to <i>specify</i> , not to implement.

READING ORDER THAT WORKS

Wirth end-to-end first (one week, it is short) so you have a complete mental model of a whole compiler. Then Appel chapter by chapter, one chapter ahead of the part you are implementing. Dragon and Pierce on demand, by section. A compiler read about is worth roughly nothing; the point of Wirth first is that it is short enough to finish before you get bored, and it will stop you over-engineering Part 4.

QUESTION TO SETTLE NOW

Are you writing this compiler to *learn compilers*, or to *have a Pascal compiler*? The answer changes the roadmap. If the former, hand-write everything (lexer, parser, register allocator later) and accept a narrower language. If the latter, generate the parser, take a wider dialect, and spend the saved time on the back ends. This document assumes the former with an eye on the latter — it is the right bias for a project whose stage 2 is “write an adaptive VM”.

Scope: which Pascal are you writing?

Undecided scope is the most common reason hobby compilers die. Fix the dialect before you write the lexer, and write the exclusions down.

“Pascal” is at least four languages: ISO 7185 (Standard Pascal, 1983), ISO 10206 (Extended Pascal, 1990), UCSD/Turbo Pascal, and Delphi / Object Pascal. They differ in ways that reach all the way into your back end — Turbo strings are length-prefixed and assignable, ISO strings are `packed array[1..n] of char` with different assignment rules; Turbo has units, ISO has one monolithic program; Turbo short-circuits boolean operators, ISO does not specify the order of evaluation at all.

RECOMMENDATION

Target **ISO 7185 level 0** (no conformant array parameters) as the specification, plus a short, explicit list of Turbo-flavoured conveniences you allow. Reason: 7185 is a real, finite, freely available standard with a conformance test suite in the wild, and “does the standard permit this?” is then always an answerable question. “What would Turbo do?” is not.

Feature triage

Put every language feature into exactly one of three buckets and keep the table in the repository. It is the single most useful artefact of the whole scoping exercise, because it turns “this program does not compile” into either a bug or a documented non-goal.

BUCKET	FEATURES
Core must work at M8	integer / real / boolean / char, enumerations, subranges, constants, <code>array</code> (multi-dimensional, arbitrary ordinal index types), <code>record</code> (fixed part), <code>set</code> , statically nested procedures and functions, value and <code>var</code> parameters, full expression language, <code>if / while / repeat / for / case / with</code> , <code>read / readln / write / writeln</code> on standard input and output, the required functions (<code>abs</code> , <code>sqr</code> , <code>ord</code> , <code>chr</code> , <code>succ</code> , <code>pred</code> , <code>odd</code> , <code>trunc</code> , <code>round</code> , <code>sin</code> ...).
Later after M8, in this order	pointers with <code>new / dispose</code> , linked structures, variant records, file variables beyond <code>text</code> , <code>goto</code> (local first, non-local later), procedural parameters, conformant arrays, Turbo-style units or an ISO 10206 module system.
Excluded write it down, with the reason	Objects/classes, exceptions, generics, inline assembler, absolute addresses, pointer arithmetic, <code>packed</code> as a storage guarantee (accept the keyword, ignore the packing), untyped <code>var</code> parameters.

Semantics you cannot reproduce exactly, and must document

Start `docs/DEVIATIONS.md` at the same time as the lexer. Every entry is a line of the form *standard says X; Pascal2J does Y; because Z*. Candidates you will hit:

- **Integer overflow.** ISO makes it an error; the JVM wraps silently. Decide: wrap (fast, wrong), or check with `Math.addExact` and friends (correct, and on HotSpot nearly free because it is intrinsified — measure it, this is Task 1.3).
- **Real arithmetic and output format.** ISO specifies a default floating output format that is not Java's `Double.toString`. Matching it exactly is a week of tedium; matching `x:w:d` field notation is a day and worth it.
- **dispose and dangling pointers.** On a garbage-collected host, `dispose` cannot make a pointer dangle. Either make it a no-op or (better for teaching) poison the cell in debug builds so use-after-dispose is detected rather than silently

working.

- **Variant records used for type punning.** The standard says reading a field of an inactive variant is an error; real programs do it deliberately. You cannot reinterpret an `int` as a `float` through a JVM object field. Decide early whether variants are a tagged union (safe, breaks punning) or you exclude them.
- **Uninitialised variables.** ISO leaves them undefined; the JVM zero-initialises everything. Your programs will be *more* defined than the standard. That is fine, but a program that relies on it is not portable — consider a definite-assignment warning (Part 10).
- **Evaluation order and short-circuiting.** See Part 11; the choice changes generated code and program meaning.

FROM YOUR OWN REPOSITORY

`testSources/ok/test0.pas` is filed under `ok/` but is not valid ISO Pascal: `var c := 'y'` has no type, `const 9n : integer = 0` is not an identifier, a typed `const` with an initialiser is a Turbo extension, `const` appears after `var`, and `procedure v();` has empty parentheses that ISO does not allow. That single file contains at least five dialect decisions. Splitting your corpus into `ok/` and `reject/` and justifying each verdict against the standard is genuinely the best first day of work on this project — it forces every question in this part to the surface before any code depends on the answers. (While you are there: rename to `testSources/.`)

TASKS

- 1.1 (1) Fill in the triage table for your grammar; any production you cannot place is a production you do not understand yet.
- 1.2 (2) Classify every file in `testSources/` as accept or reject, with a one-line standard citation for each reject. Design the annotation format now — e.g. a comment `{! expect error E0102 at 3:8 !}` — because Part 4 will consume it.
- 1.3 (1) Write a JMH microbenchmark comparing `a + b` against `Math.addExact(a, b)` in a hot loop. The result will surprise you, and it decides your overflow policy with data instead of folklore.

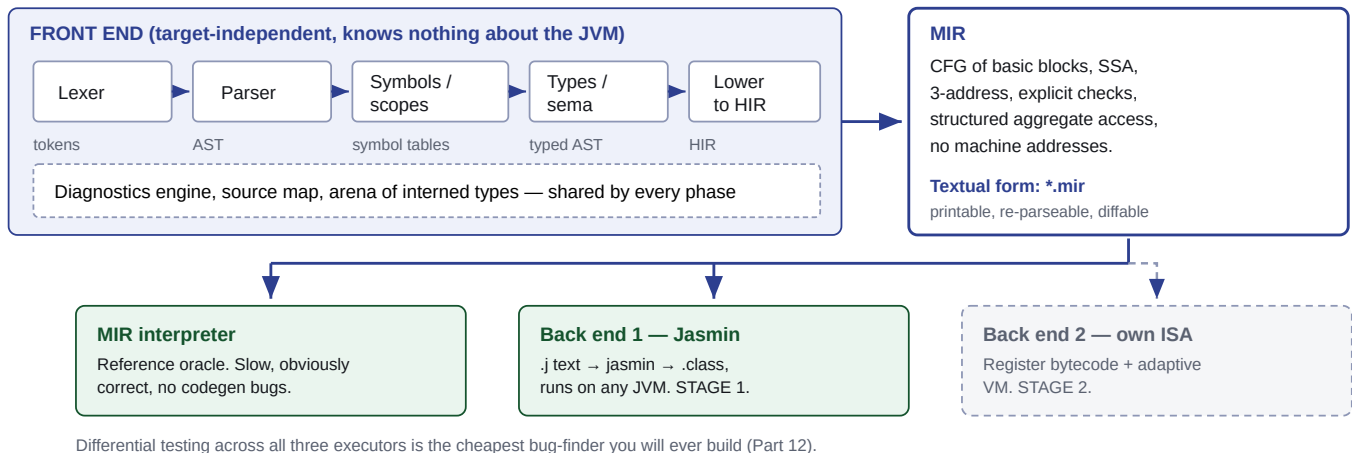
QUESTIONS

Does your grammar allow the ISO declaration order (`label`, `const`, `type`, `var`, procedures, in that order, each at most once) or the Turbo free order? — Which decision makes your parser simpler, and is “simpler parser” a good reason to restrict a language? — If you accept both, where does the diagnostic live: in the parser or in a later check, and which gives the better error message?

Architecture and the front/back boundary

You asked for a shared front end and a replaceable back end. That is the right shape, and it is worth being strict about: the boundary is the deliverable of stage 1, not the `.class` files.

The pipeline



The boundary that matters is MIR, and it is worth the extra cost of a printable, re-parseable textual form.

Why two IR levels, and not zero or three

Wirth uses none: his parser calls the code emitter directly. That works because Oberon-0 is designed for it and he does no optimisation. Appel uses one tree IR plus a canonicalised linear form. Dragon describes three-address code. For you:

- **HIR** — a typed, desugared tree. Names resolved, every implicit conversion made explicit, `for` rewritten, with eliminated, constant expressions folded. Still structured: loops are loops. It is the last form in which a source-level debugger or a pretty-printer is easy.
- **MIR** — a control-flow graph of basic blocks over three-address instructions, in SSA form, with an explicit check vocabulary (range check, nil check, index check, overflow check) so the optimiser can *remove* them. This is what both back ends consume.

Resist a third level. The temptation is a “LIR” close to JVM bytecode; you do not need it, because the JVM is a stack machine and post-order traversal of MIR expressions already gives near-optimal stack code.

THE ONE NON-OBVIOUS RECOMMENDATION IN THIS DOCUMENT

Make MIR **printable and re-parseable** from day one: `pascal2j --emit=mir foo.pas` writes `foo.mir`, and `pascal2j --from-mir foo.mir --backend=jasmin` reads it back. It costs perhaps two days. In return: your back end gets unit tests that do not involve Pascal at all, your optimiser gets golden-file tests that a human can read in a diff, you can hand-write pathological MIR to test the back end, and in stage 2 you can develop your VM against `.mir` files without the front end in the loop. Every serious compiler has this (LLVM’s `.ll`, Cranelift’s CLIF) and every student compiler that lacks it ends up debugging the front end and back end simultaneously.

Enforcing the boundary

“Separate front end” is a property you have to defend mechanically, because the first deadline will tempt you to sneak a JVM detail into the type checker (a descriptor string, a slot number, a `java/lang/String`). Make it impossible:

- Separate Gradle modules, with `:frontend` having no dependency on `:backend-jasmin` — the compiler then rejects the shortcut for you.
- The driver depends on both and wires them via an interface:

```
interface Backend {  
    val id: String // "jasmin", later "dvm"  
    fun emit(program: MirProgram, opts: BackendOptions): List<Artifact>  
}
```

- A negative test: `./gradlew :frontend:check` must pass with `:backend-jasmin` absent from the build entirely.

QUESTIONS

What, exactly, belongs to the front end when the back ends differ this much? Integer widths, for instance: is “`integer` is 32-bit two's complement” a language fact (front end) or a target fact (back end)? — Where does the decision to represent `set of char` as four 64-bit words live? — If stage 2's VM is register-based and untagged, does MIR need a notion of value size, and if you add one, is MIR still target-independent or have you smuggled a target in?

TASKS

2.1 (2) Write the MIR data types and its printer *before* writing the code that produces MIR. Hand-write three `.mir` files by hand (a loop, a call, an array store) and make the printer round-trip them.

2.2 (1) Draw your own version of the diagram above, and for each arrow write down what data crosses it and what invariants hold there. Put it in `docs/`. When you break an invariant in four months, this drawing is what tells you which side is wrong.

Milestone map

Thirteen milestones. The order is chosen so that you always have a working compiler for *some* language, and it grows. Never spend three months before the first program runs.

Effort is given in weeks at roughly ten focused hours a week, for someone learning the material as they go. They will be wrong; the ratios between them will not be.

M	MILESTONE	EXIT CRITERION — A BINARY, TESTABLE STATEMENT	EFFORT
M0	Scope and corpus	Triage table filled in; <code>testSources/</code> split into accept/reject with citations; <code>1 DEVIATIONS.md</code> exists.	1
M1	Vertical slice	<code>begin writeln(42) end.</code> compiles through lexer → parser → HIR → MIR → Jasmin and the resulting <code>.class</code> prints 42 under <code>java .</code> Everything is a stub; nothing is general.	1–2
M2	Diagnostics infrastructure	Every error has a code, a span, and a rendered caret; the reject corpus asserts exact codes and positions; a fuzzer cannot make the compiler throw an uncaught Kotlin exception.	1
M3	Full lexer	Every token in your grammar, case-insensitive, both comment forms, <code>1..3</code> handled, string escapes, all literal forms; golden token dumps for the corpus.	1
M4	Full parser + AST	Whole grammar parses; FIRST/FOLLOW conflicts documented; error recovery produces at most one diagnostic per broken statement and never loops; AST printer round-trips.	2–3
M5	Symbols and scopes	Nested procedures resolve correctly, including forward declarations and shadowing; a scope dump test; every identifier in the AST has a resolved symbol or a diagnostic.	1–2
M6	Type checker	Your written typing rules are implemented one-to-one; assignment compatibility, ordinal indexing, set base types, <code>var</code> -parameter lvalue rules, function result assignment. Reject corpus fully green.	3–4
M7	HIR + MIR + interpreter	The MIR interpreter runs the whole accept corpus and produces the expected output. <i>You now have a working Pascal implementation with no back end.</i>	2–3
M8	Jasmin back end, correct	Every accept-corpus program produces byte-identical output under the JVM and under the MIR interpreter. No optimisation whatsoever. This is the real stage-1 goal.	3–4
M9	Runtime library + I/O fidelity	<code>write/writeln</code> field widths, reals, <code>read/readln</code> tokenising, <code>eof/eoln</code> ; runtime errors carry Pascal line numbers in the JVM stack trace.	1–2
M10	Benchmarks + measurement	Five benchmark programs, a harness with warm-up, and a committed baseline table. <i>Before</i> any optimisation exists.	1
M11	Optimiser	SSA construction, SCCP, DCE, GVN, LICM, range analysis and check elimination. Each pass ships with: a golden MIR test, a correctness argument, and a measured delta on M10's benchmarks — including the ones where it measures zero.	4–6

M12	Stage 2 spike	Own ISA specified, assembler/disassembler, verifier, and a plain interpreter that runs the corpus. Only then: profiling, tiering, OSR, deopt.	6+
------------	---------------	---	----

ON M1, THE VERTICAL SLICE

Do not build the lexer completely, then the parser completely. Build the thinnest possible path through all six phases first, with a lexer that knows five tokens and a back end that can only emit `println(int)`. You will discover the shape of every interface in a week rather than in month four, when changing them is expensive. This is the single highest-leverage scheduling decision on the list, and it is the one most students skip.

ORDERING TRAPS

M10 before M11 is not negotiable. An optimiser without a benchmark harness is a hobby, and on the JVM specifically you will find that several textbook passes measure exactly zero because HotSpot's C2 already did them. That is a *result*, not a failure — but only if you measured. — Likewise M7 before M8: the MIR interpreter seems like a detour until the first time your generated bytecode is wrong and you can ask “is the front end or the back end lying?” and get an answer in one command.

QUESTIONS

Which milestone is the first one where you could stop and still have something you are proud of? — Which are you most likely to skip, and what would it cost? — M6 is the longest single milestone; can you split it so that something is demonstrable every week?

Foundations: sources, spans, diagnostics

The quality of a compiler, as experienced by its users, is almost entirely the quality of its error messages. Build the machinery before the phases that need it, or you will retrofit spans through fifty classes.

GOAL

A diagnostics engine that every later phase reports through, and a source map that turns byte offsets into human positions.

DESIGN DECISIONS

- **Spans, not line/column.** Store `(startOffset, endOffset)` as two `Int`s in every AST node; compute line and column lazily from a line-start index built once per file. Line/column in every node is memory you pay for and a conversion you repeat.
- **Stable error codes.** `E0102`, not a message string. Codes let tests assert on meaning rather than wording, let you add `--explain E0102` later, and let you change the prose freely.
- **Structure, not strings.** A diagnostic is `{severity, code, primarySpan, message, notes[], helps[]}`. Rendering is a separate concern, so you can add JSON output for an IDE without touching any phase.
- **Fail-soft.** Phases return partial results plus diagnostics; only the driver decides whether to continue. An `error` type and an `error` symbol that absorb further errors silently are what stop one mistake producing forty messages (Appel ch. 5 makes this point about semantic analysis specifically).
- **Determinism.** The compiler must be a pure function from inputs to outputs, including the *order* of diagnostics. Use `LinkedHashMap`, never iterate a `HashSet` into output, sort by span before rendering. Golden tests will punish you otherwise, non-reproducibly, on someone else's machine.

TASKS

4.1 (1) Implement the caret renderer: source line, underline of the primary span, notes beneath. Handle tabs (a tab is not one column) and multi-line spans.

4.2 (1) Build the test harness that reads `{! expect E0102 at 3:8 !}` annotations out of a `reject/` file and asserts the compiler produced exactly that set — no more, no fewer. The “no more” half is what catches cascading-error regressions.

4.3 (2) Add `--emit=tokens|ast|hir|mir|jasmin` to the driver on day one. Every phase becomes independently inspectable, and every bug report becomes reproducible with one command.

QUESTIONS

When is it right to stop after a phase and when should you push a partial result forward? (A parse error usually stops semantic analysis; a *type* error usually should not stop checking the rest of the program.) — What is your policy for the second and subsequent errors caused by the same root mistake, and how will you test that the policy holds?

Reading

Appel ch. 4 §“abstract syntax positions” and ch. 5 on error types absorbing cascades.

Wirth ch. 7 — his whole error strategy in two pages; note how cheap it is and how much it buys.

Dragon §1.3, §4.1.3–4.1.4 on error-recovery strategies and their trade-offs.

Lexical analysis

The easiest phase, with three genuine traps in Pascal specifically. Hand-write it; a generator buys you nothing at this size and costs you control of error messages.

DELIVERABLE

A lexer producing a token stream with spans, plus golden token dumps for the corpus.

THE THREE PASCAL TRAPS

- **1..3 versus 1.5.** Pure maximal munch lexes `1..3` as the real number `1.` followed by `.3`, and your own test file contains `array [1..3]`. The fix is one character of extra lookahead: on a digit sequence followed by `.`, peek the character after the dot; if it is another `.`, emit the integer and leave both dots. This is the textbook example of why maximal munch needs a backtracking escape hatch, and it is worth understanding as theory, not just as a patch — the same issue is why C needs `>>` disambiguation in templates and why Pascal's designers regretted nothing.
- **Case insensitivity.** `Begin`, `BEGIN` and `begin` are one keyword; `Foo` and `foo` are one identifier. Fold for lookup but *keep the original spelling* for diagnostics, so the error says `Foo` the way the user wrote it. In Kotlin, fold with `lowercase()` and be aware that a locale-sensitive fold turns `I` into a dotless `ı` in Turkish locales; `lowercase()` on Kotlin is locale-independent (it is `Locale.ROOT` semantics) but `String.toLowerCase()` from Java is not. A compiler whose behaviour depends on the machine's locale is a real bug that has shipped in real products.
- **Comments.** `{ ... }` and `(* ... *)` are both comments, and in ISO Pascal a comment opened with `{` may be closed with `*`. They do not nest (Turbo optionally nests them). An unterminated comment at end of file must produce a diagnostic pointing at the *opening* brace, not at EOF — that single detail is the difference between a five-second fix and a ten-minute hunt for the user.

Minor but real: `$FF` or `16#FF` hex forms depending on dialect; character literals `#13`; string literals with `'` as an embedded apostrophe; the fact that a one-character string literal `'a'` is a `char` in Pascal, which makes literal typing context-dependent — a decision you should take in Part 8, not here.

TASKS

- 5.1 (1)** Write the DFA for numeric literals on paper before coding: integer, real with fraction, real with exponent, and the `..` escape. Then implement it and check your implementation against the drawing. Dragon §3.4 is the technique.
- 5.2 (1)** Property test: for a randomly generated token sequence, printing the tokens with a single space between them and re-lexing must yield the same sequence. This catches an entire class of lexer bug in twenty lines.
- 5.3 (2)** Measure: lex a 100k-line synthetic file. If you are slower than about 10 MB/s you are probably allocating a `String` per token; intern identifiers into a symbol table of `Int` ids instead and measure again. (Worth doing once, for the lesson in how compilers actually spend time.)

QUESTIONS

Should the lexer or the parser decide that `'a'` is a `char` rather than a string? — Do you emit comments and whitespace as trivia attached to tokens (needed later for a formatter or an IDE) or discard them (simpler now)? What does that choice cost if you change your mind at M6? — Is the keyword/identifier distinction made in the DFA or by a table lookup after it, and which one does Wirth use, and why?

Reading

Dragon ch. 3 entire — §3.3 regular expressions, §3.4 recognition, §3.6–3.7 NFA → DFA and minimisation. Read §3.5 on Lex only to decide not to use it.

Wirth ch. 3 (regular languages) and the scanner in ch. 7 — roughly 60 lines of Oberon, for a complete language.

Appel ch. 2 — particularly the discussion of why longest-match plus rule priority is the right specification.

Parsing and the AST

Pascal was designed to be parsed by recursive descent by a human with a pencil. Take the gift.

DESIGN DECISIONS

- **Recursive descent** for declarations and statements, **precedence climbing** (Pratt) for expressions. Writing `factor/term/simpleExpression/expression` as four mutually recursive functions, the way the grammar is written, is also fine and is what Wirth does; precedence climbing is the same thing with the level as a parameter. Do the naive version first, then refactor to Pratt and notice they produce identical trees — that refactor is a good half-day of insight.
- **An LL(1) check, mechanically.** Do not *assume* your grammar is LL(1). Compute FIRST and FOLLOW for every production with a small script over your grammar file and print the conflicts. Every conflict is either a bug in your grammar or a place your parser needs extra lookahead — and you want that list explicitly, not discovered one at a time by failing tests.
- **Error recovery: FOLLOW-set panic mode.** Wirth's technique (ch. 7) is to pass each parsing procedure the set of tokens that may legally follow it, and to skip input until a token in that set appears. It is about thirty lines and gives good behaviour. Appel §3.4's Burke–Fisher local repair is better and considerably more work; read it, implement it later if at all.
- **AST shape.** Sealed interfaces and immutable data classes in Kotlin, one node type per construct, every node carrying a `NodeId` and a span. **Keep the results of later phases in side tables keyed by `NodeId`** (`types: Map<NodeId, Type>`, `symbols: Map<NodeId, Symbol>`) instead of mutable fields on nodes. You get immutability, you can run a phase twice, you can throw a phase's results away, and you never wonder whether a field has been filled in yet. The cost is one indirection per lookup, which is free at this scale.

TRAPS

Dangling else — the grammar is ambiguous; the rule is “bind to the nearest unmatched `if`”, which recursive descent gives you for free simply by consuming `else` greedily in the inner call. Know that you relied on an implementation accident and write the test.

Error recovery that does not consume input loops forever. Invariant: every iteration of a recovery loop either consumes a token or exits. Assert it.

Semicolons. Pascal's `;` is a separator, not a terminator, so `begin a; b; end` contains an empty statement — legal, and the source of the classic “`;` before `else`” error. Decide whether to warn.

TASKS

6.1 (2) Write the FIRST/FOLLOW calculator over your own grammar file and commit the conflict report as a golden test. When you extend the grammar later, a new conflict shows up in a diff.

6.2 (1) Take twenty programs from your corpus, delete one token from each at random, and require exactly one diagnostic and a complete parse tree for the rest of each file. This is the fastest way to find out whether your recovery is real.

6.3 (2) Implement an AST pretty-printer, then a property test: `parse(print(parse(src))) == parse(src)`. Cheap, and it catches precedence and associativity errors that eyeball tests miss.

6.4 (1) Ambiguity hunt: find two different grammars for Pascal expressions, one ambiguous and one not, and explain in a paragraph why precedence climbing resolves the ambiguity without changing the grammar.

QUESTIONS

Do you want a concrete syntax tree (every token, including punctuation and comments) as well as an abstract one? It is what makes a formatter, an IDE and refactoring possible, and it roughly doubles the size of Part 6. What is it worth to *this* project? — Where do you put the boundary between “the parser rejects it” and “the parser accepts it and a later check rejects it”? Consider `const 9n = 0`: is that a lexical error, a syntax error, or a semantic one, and which produces the most helpful message?

Reading

Wirth ch. 2, 4, 7 — deterministic parsing conditions and the complete Oberon-0 parser. The clearest exposition of recursive descent in print.

Dragon §4.4 (predictive parsing, FIRST/FOLLOW, tables), §4.4.5 (panic-mode recovery). Skim §4.5–4.9 (LR) for literacy; you will not write an LR parser here.

Appel ch. 3 (esp. §3.2 predictive parsing and §3.4 error recovery) and ch. 4 (abstract syntax — note his separation of abstract from concrete syntax, and the visitor question).

Pierce ch. 3 — not about parsing, but the place to learn the inductive-definition style you will use to specify the AST and prove things about traversals.

Scopes, symbols and nesting

Pascal's nested procedures are its defining feature and the place where the JVM will fight you hardest. Decide the strategy here, in the front end, and make the back end's job mechanical.

DELIVERABLE

Every identifier occurrence resolved to a symbol, or diagnosed. A dump format for scopes that you can golden-test. A computed *escape* flag on every local variable.

The problem

A procedure nested inside another can read and write the outer procedure's locals:

```
procedure Outer;
var total: integer;           { lives in Outer's frame }
  procedure Add(x: integer);
  begin
    total := total + x        { non-local access, one level up }
  end;
begin total := 0; Add(5); writeln(total) end;
```

On a conventional machine you solve this with a **static link** (a pointer in each frame to the frame of the lexically enclosing procedure) or a **display** (an array indexed by nesting depth). Wirth ch. 12 and Dragon §7.3 both cover this; Appel ch. 6 is the most complete treatment and is where you should start.

The JVM gives you neither: a method has locals, and nothing else can reach them. Your options:

STRATEGY	WHAT IT MEANS	COST
Heap frame objects	Each procedure whose locals are captured gets a generated class <code>Outer\$Frame</code> with a field per captured variable; inner procedures become static methods taking the frame as an extra parameter, plus its own parent chain for deeper nesting.	One allocation per activation of an enclosing procedure; field access instead of local access. Simple, uniform, obviously correct.
Lambda lifting (closure conversion)	Add each captured variable as an extra parameter to the inner procedure, and thread it through all call sites. Classic; see Appel ch. 15.	Only works directly for variables that are read, not assigned — an assignment must be visible to the outer frame, so a mutable captured variable still needs a cell. Parameter lists can explode under deep nesting.
Everything on the heap	Every local of every procedure becomes a field of a frame object.	Trivially correct, uniformly slow, and it defeats HotSpot's register allocation. Do not.

RECOMMENDATION

Heap frame objects, **plus an escape analysis so that only actually-captured variables move**. In practice the overwhelming majority of locals in real Pascal are never touched by an inner procedure, so they stay in JVM locals where HotSpot can put them in registers. Escape analysis here is ten lines — walk the body of each nested procedure, mark every non-local symbol it references — and it is the same analysis you need for `var` parameters (Part 8) and again in stage 2 for scalar replacement. It is the highest value-per-line of code in the whole front end.

TASKS

7.1 (1) Implement the escape marking and print a report: for your corpus, what fraction of locals escape? (Predict the number before you run it. The gap between your prediction and the result is the lesson.)

7.2 (2) Implement the parent-chain walk for a variable referenced three levels up, and write the test program that exercises it — including recursion in the middle procedure, which is where a naive implementation that keeps “the” frame of a procedure in a static field breaks.

7.3 (1) Read Appel §6.2 on displays and write half a page on why a display is faster than a static chain, and why nobody uses one today. (Hint: it is about recursion and about how deep real nesting actually goes.)

QUESTIONS

Pascal permits a procedure to be passed as a parameter (a procedural parameter). If you allow it, a procedure value is a pair (code, frame) — a closure. Does your strategy still work, and does the answer change which strategy you should pick *now*? — What happens to the frame object when the outer procedure returns while an inner procedure is still reachable? In Pascal it cannot be (no upward funargs), so the frame is stack-allocatable in principle — can you express that fact to the JVM, and does HotSpot's escape analysis find it for you? — How do you handle a **forward** declaration: one symbol with two declaration sites, or two symbols that must be unified?

Reading

Appel ch. 6 entire (frames, static links, escape analysis — his **escape** pass is exactly Task 7.1), ch. 15 for closure conversion.

Wirth ch. 12 — procedures and the concept of locality; the static link done minimally.

Dragon §7.1–7.3 — activation records, stack allocation, access to non-local data. §7.3.5–7.3.8 covers displays.

Pierce ch. 6 (nameless terms / de Bruijn indices) — the theoretical version of “a variable is a nesting depth plus an offset”, which is literally your static-link representation.

Types and semantic analysis

The longest milestone, and the one where Pierce earns his place on your shelf. Write the rules down in inference-rule notation first; implement them second; the implementation then has a specification to be wrong against.

DELIVERABLE

A type for every expression, a diagnostic for every ill-typed program in the reject corpus, and a document — half a dozen pages — containing your typing rules.

Write the rules down

Pascal is simply typed, so the whole type system fits in a few pages of judgements in Pierce's notation. Start with the environment and the shape of the judgement:

$\Gamma \vdash e : \tau$	"in context Γ , expression e has type τ "
$\Gamma \vdash s \text{ ok}$	"in context Γ , statement s is well formed"
(T-Index)	$\frac{\Gamma \vdash a : \text{array}[\iota] \text{ of } \tau \quad \Gamma \vdash i : \iota' \quad \iota' \leq \iota}{\Gamma \vdash a[i] : \tau}$
(T-Assign)	$\frac{\Gamma \vdash l : \tau_1 \quad \text{lvalue}(l) \quad \Gamma \vdash e : \tau_2 \quad \tau_2 \leq \tau_1}{\Gamma \vdash l := e \text{ ok}}$

The interesting content is entirely in the relation written $\leq$ — Pascal's **assignment compatibility**, which is *not* type identity and *not* subtyping in Pierce's sense. It says roughly: identical types are compatible; `integer` is compatible with `real` (with an implicit widening); a subrange is compatible with its host type and vice versa *subject to a runtime check*; a set type is compatible with another set type whose base types are compatible; strings of equal length are compatible. Writing this relation out precisely, as rules, is the single most valuable half-day in Part 8 — and it will expose at least three cases you had not thought about.

WHY THIS IS NOT SUBTYPING

In a subtyping system (Pierce ch. 15) $S <: T$ means every value of S is a value of T , and coercion is free. Pascal's subrange compatibility is different: assigning `integer` to `0..9` is permitted by the type system but may *fail at runtime*. It is a checked coercion, closer to a cast with a proof obligation than to subsumption. Notice what this means for you: the type checker's job is not only to accept or reject, but to **record where a runtime check is required** — and that record is what the optimiser will later try to discharge statically (Part 10). Make the check an explicit node in HIR at the moment the type rule licenses it; do not leave it implicit for the back end to remember.

The checklist

Semantic analysis is a long list of small rules, and the way to not miss any is to keep the list. At minimum: undeclared identifier; duplicate declaration in one scope; type identity for `var`-parameter arguments (compatibility is *not* enough — a `var` parameter requires an identical type and an lvalue argument); assignment to a `const`; assignment to a `for`-loop control variable inside its own loop (ISO forbids it; a nice check to implement because it needs a small scope-like stack of “currently controlled” variables); a function that never assigns its result; argument count and kind; `case` label duplication and type agreement with the selector; set element ordinality and base range; array index type must be ordinal and bounds must be constant; enumeration constant scope leakage; `with`-statement field injection and its shadowing rules; the required-procedure special cases.

WRITE AND WRITELN ARE NOT PROCEDURES

They are variadic, they accept arguments of several unrelated types, they accept the `value : width : digits` syntax that appears nowhere else in the grammar, and `writeln` takes an optional first argument that is a file. No user procedure can be declared like that. Do not try to make them fit your ordinary call-checking path; give them a dedicated checker and a dedicated HIR node. Every Pascal implementation does this, and discovering why is worth an hour of trying not to.

TYPE IDENTITY IN PASCAL IS BY NAME, MOSTLY

`type A = array[1..10] of integer; B = array[1..10] of integer;` — are A and B the same type? In ISO Pascal, no: they are distinct types and not assignment compatible, though they are structurally identical. But an anonymous type written inline in two `var` declarations is also two distinct types, which is why `var x, y: array[1..10] of integer;` makes `x` and `y` compatible (one declaration, one type) but splitting the line into two makes them incompatible. Implement types as interned objects with identity, created once per syntactic type denoter, and this falls out for free. Get it wrong and you will rediscover it via a very confusing bug.

TASKS

8.1 (2) Write out the assignment-compatibility relation as inference rules, then implement `assignable(from: Type, to: Type): Compatibility` returning `ok / ok-with-check / ok-with-conversion / no`. Property test: generate random type pairs and assert the implementation agrees with a second, deliberately naive, direct transcription of the rules.

8.2 (2) Build the compile-time constant evaluator as its own module returning a `ConstValue`. You need it for array bounds, subrange bounds, `case` labels and `const` declarations — and you will reuse it verbatim as the constant-folding core of your optimiser. Getting that reuse is the point of the exercise.

8.3 (3) Pierce ch. 8: state and prove *progress* and *preservation* for a tiny fragment of your language (integers, booleans, variables, assignment, `if`, `while`). Write the small-step operational semantics first. This is the exercise that converts “my type checker seems to work” into “I know what my type checker guarantees” — and the answer for Pascal is interesting, because with subranges and uninitialised variables you *cannot* prove the usual progress theorem. Working out precisely which hypothesis fails is the lesson.

8.4 (1) Pierce ch. 13 is about references. Read it, then write a paragraph mapping Pascal's `var` parameters and `^T` pointers onto his `Ref T / ! / :=` — including what his store typing corresponds to in your compiler, and why Pascal's restriction that you cannot take the address of an arbitrary expression makes your life easier than his.

QUESTIONS

Is `packed` part of a type's identity? (ISO says it is.) — If `integer` is 32-bit and `real` is 64-bit, is `integer → real` lossless? For 64-bit integers it would not be; what does your standard say, and what will you do when stage 2's VM has different word sizes? — Do you allow a subrange of an enumeration as an array index type, and what is the resulting array's length? — What is the type of `'a'`, and of `'ab'`, and what does that imply about assigning `'a'` to a `packed array[1..1] of char`?

Reading

Pierce ch. 3 (inductive definitions, operational semantics, the proof style), ch. 8–9 (typed arithmetic; STLC; progress and preservation), ch. 11 (§11.7–11.9 records and variants; §11.12 general recursion), **ch. 13 (references)**, ch. 14 (exceptions — your runtime errors), ch. 15–16 (subtyping, and why Pascal's relation is not it), ch. 19 (Featherweight Java — read before Part 11).

Appel ch. 5 — symbol tables, type representation, and the pragmatics of reporting type errors without cascading.

Wirth ch. 5 (attributed grammars) and ch. 8 (context specified by declarations) — the pragmatic minimum, and a useful counterweight to Pierce's rigour.

Dragon ch. 5 (syntax-directed definitions — the right framework for structuring these passes), §6.3–6.5 (types and declarations, type checking, coercions).

HIR: desugaring and lowering

A short phase that pays for itself by deleting language features. Everything you desugar here is a feature neither back end ever has to know about.

WHAT HIR REMOVES

- `for` loops → explicit initialise / test / body / increment (see the trap below — this is not as easy as it looks).
- `with` statements → a temporary holding the record reference, and fully qualified field accesses.
- `repeat` → a loop with the test at the bottom; `case` → a multiway-branch node on an ordinal, with the labels normalised to integers.
- Implicit conversions → explicit `Convert` nodes; implicit range checks → explicit `Check` nodes; string literal padding; set construction from element lists.
- Nested procedures → flat list of procedures with explicit frame parameters (the decision from Part 7 is *applied* here).
- `var` parameters → explicit cell/reference passing.
- Named constants → literals; constant expressions → values.

THE FOR LOOP IS A TRAP, AND A GOOD EXERCISE

Pascal's `for i := a to b do S` is not C's `for`. (1) `a` and `b` are evaluated **once**, before the loop. (2) If `a > b` the body never executes. (3) The control variable is undefined after the loop — you may not read it. (4) The body may not assign to it. And the one that catches everyone: (5) if `b` is the largest value of the control variable's type, the obvious lowering `i := a; while i <= b do begin S; i := i + 1 end` **overflows or loops forever** on the last iteration. The correct shape is a bottom-tested loop guarded by an initial `a <= b` test, with the increment guarded by `i <> b`:

```
if a <= b then begin
  i := a;
  loop: S; if i = b then goto done; i := succ(i); goto loop;
  done:
end
```

Write the test with `b = maxint` before you write the lowering. Then check what your favourite real compiler does with it.

QUESTIONS

Is HIR a separate data type, or is it “the AST with some node kinds forbidden”? The first costs you a second tree definition and a translation; the second costs you an invariant that is not checked by the type system. Which mistake would you rather debug? — Should HIR be typed (every node carries its `Type`) or is the type information consumed and discarded by lowering? Consider stage 2, where an adaptive VM might want source types for specialisation. — Does any optimisation belong at HIR level rather than MIR? (Consider: `with`-common-subexpressions, string-literal interning, and inlining of trivially small procedures — inlining before closure conversion can be much more effective.)

Reading

Appel ch. 7 — translation to intermediate representation; the discussion of why an IR tree is simpler than the AST and what “lower” means.

Dragon §6.1 (variants of syntax trees), §6.4 (translation of expressions), §6.9 (intermediate code for procedures).

MIR: control-flow graph, SSA, and the optimiser

The part of the project where you will learn the most, and the part where it is easiest to waste six months producing no measurable improvement. Both facts are worth taking seriously.

The representation

A program is a list of procedures; a procedure is a set of basic blocks with a single entry; a block is a list of instructions ending in exactly one terminator. Instructions are three-address and define at most one value. In SSA form each value is defined exactly once and blocks begin with ϕ -nodes.

```
proc Sum(n: i32) -> i32 {
  bb0:
    v0 = const 0
    br bb1
  bb1:
    v1 = phi [v0, bb0], [v4, bb2]    ; this first, always
    v2 = phi [const 1, bb0], [v5, bb2]; total
    v3 = icmp.sle v2, n
    br v3, bb2, bb3
  bb2:
    v4 = add.ovf v1, v2              ; checked add: traps on overflow
    v5 = add v2, 1
    br bb1
  bb3:
    ret v1
}
```

FOUR DECISIONS THAT WILL MATTER LATER

- **Safety checks are instructions, not implicit behaviour.** `check.range v, lo, hi`, `check.index a, i`, `check.nil p`, `add.ovf`. If a check is implicit in the semantics of `store`, no pass can ever remove it. If it is an instruction, removing it is just dead-code elimination once an analysis proves the condition.
- **Aggregates stay structured.** No address arithmetic, no `load [base + 4*i + 8]`. Use `elem a, i` and `field r, 3`. Appel and Dragon both lower to address arithmetic because they target real machines; the JVM cannot express it, and your stage-2 VM does not have to either. This is the main place where you should deliberately deviate from the books, and knowing *why* you are deviating is the point.
- **Keep a source position and a live-variable map on every instruction that can trap.** You do not need them in stage 1 beyond error messages. In stage 2 they become your deoptimisation metadata, and retrofitting them into an optimiser that has already destroyed the mapping is genuinely painful. This is the cheapest thing you can do now to make stage 2 possible later.
- **SSA construction algorithm.** Cytron et al.'s dominance-frontier algorithm is what Appel ch. 19 and Dragon §9.5.1 teach, and you should understand it. But for a hand-written compiler, Braun et al. (2013), "Simple and Efficient Construction of SSA Form", builds SSA directly during IR construction with no dominance computation at all, in about 150 lines. Recommendation: implement Cytron once as a learning exercise, then use Braun in the real pipeline, and write down which you would choose for a production compiler and why.

Passes, and whether they pay

PASS

WHAT IT BUYS YOU ON THE JVM

PRIORITY

Sparse conditional constant propagation (SCCP)	Folds constants <i>and</i> prunes unreachable branches in one fixpoint; strictly stronger than doing either alone. Feeds every other pass.	High
Dead code elimination	Mostly cleans up after other passes; also removes your own redundant checks once they are proved.	High
Range / value analysis + check elimination	The big one. Subrange assignments and array indexing generate checks that HotSpot cannot remove because it does not know your language's invariants. This is a pass that is <i>yours to win</i> .	High
Global value numbering / CSE	Real but modest; C2 does local CSE well. Wins where your lowering created redundancy the source did not have (index computations, frame accesses).	Medium
Inlining (small procedures, after flattening)	Enables everything else, and lets you delete frame objects. C2 also inlines — but only what it can see as hot, and only after warm-up.	Medium
Loop-invariant code motion	Measure it. On HotSpot, expect close to zero for simple loops. Implement it anyway — you need it for stage 2 — but let the measurement teach you something about where optimisation belongs in a two-tier system.	Medium
Self-tail-call → loop	The JVM has no tail calls, so a deeply recursive Pascal program overflows the Java stack. Converting direct self-recursion into a loop is a correctness-adjacent win.	Medium
Peephole / algebraic simplification	Cheap, satisfying, and almost entirely redundant with what C2 does. Good for learning; do not expect numbers.	Low
Register allocation	Not needed for the JVM at all (it has an operand stack and numbered locals). You still want liveness for local-slot reuse and for stage 2.	Stage 2

THE LESSON HIDING IN THIS TABLE

You are compiling to a platform that already has a world-class optimising compiler underneath it. That makes stage 1 an unusually honest laboratory: any pass you write either beats C2 at something C2 cannot know (language-level invariants, checks, allocation, frames) or it measures as noise. Do the measurements, write the numbers in the repository, and you will understand the division of labour between a static compiler and a JIT far better than any chapter can teach you — which is precisely the knowledge stage 2 requires.

TASKS

10.1 (2) Implement dominators (the simple iterative Cooper–Harvey–Kennedy algorithm is a page long and fast enough), then dominance frontiers, then Cytron SSA. Verify with an SSA checker: every use dominated by its def, every ϕ operand count equal to the predecessor count.

10.2 (2) Implement SCCP. Then build the test that matters: the optimised and unoptimised programs must produce identical output on the whole corpus (*metamorphic testing* — no oracle needed). Wire it into CI; it will catch more optimiser bugs than any hand-written test you write.

10.3 (3) Range analysis over SSA, then check elimination. Measure: how many of the checks emitted by the front end can you remove on your benchmarks? Report the percentage. This single number is the best summary of your optimiser's value.

10.4 (1) Take one benchmark, dump HotSpot's own output with `-XX:+UnlockDiagnosticVMOptions -XX:+PrintAssembly` (with `hsdis`) or at least `-XX:+PrintCompilation -XX:+PrintInlining`, and see what C2 did to your bytecode. Spending an afternoon here will change how you think about your own optimiser.

QUESTIONS

What is your pass ordering, and why? (There is no optimal order; there is the “phase-ordering problem”. Dragon §9.x and Muchnick discuss it.) — Should passes run to a fixpoint, and how do you stop a pass pair oscillating? — How do you know a pass is correct? Metamorphic testing catches miscompilation, but what catches a pass that silently does nothing? (Answer: a golden MIR test per pass — and this is where the textual IR from Part 2 pays for itself again.) — Your checks trap. Does that make them side-effecting, and therefore unmovable by LICM? What extra information would let you hoist a check out of a loop safely?

Reading

Appel ch. 17 (data-flow analysis), 18 (loop optimisations, dominators, induction variables), **19 (SSA — the core chapter for this part)**, ch. 8 (basic blocks and traces) for CFG construction and block ordering.

Dragon §6.2.4 (SSA introduced), ch. 8 §8.4–8.5 (flow graphs, DAG representation of blocks), §8.7 (peephole), ch. 9 entire — §9.2–9.3 data-flow frameworks and their lattices, §9.4 constant propagation, §9.5 partial redundancy, §9.6 loops and dominators.

Wirth final chapter on optimisation and front/back-end structure — deliberately modest; read it as a counterweight when you are tempted by a fourth analysis pass.

Beyond the books: Cytron et al. 1991 (SSA); Braun et al. 2013 (simple SSA); Wegman & Zadeck 1991 (SCCP); Muchnick, *Advanced Compiler Design and Implementation*, for anything the above leave vague.

The Jasmin back end

A stack machine is the easiest code-generation target in existence, right up until the verifier disagrees with you. Most of this part is about making the verifier your test oracle instead of your enemy.

DELIVERABLE

A back end that turns MIR into `.j` text, plus the toolchain step that assembles it, plus a runtime library in Kotlin that the generated code calls.

The shape of the output

```
.bytecode 49.0                                ; see the verifier trap below
.source hello.pas
.class public Hello
.super    java/lang/Object

.method public static main([Ljava/lang/String;)V
    .limit stack 2
    .limit locals 1
    .line 1
    getstatic    java/lang/System/out Ljava/io/PrintStream;
    bipush      42
    invokevirtual java/io/PrintStream/println(I)V
    return
.end method
```

Program → one public class. Global variables → static fields. Procedures and functions → static methods (they are all statically bound; you have no objects). Nested procedures → static methods taking a frame parameter, per Part 7. The main program body → `main([Ljava/lang/String;)V`.

THE VERIFIER TRAP, AND WHY `.BYTECODE 49.0` IS ON THE FIRST LINE

Class files of version 50 (Java 6) and above are verified by the *type-checking* verifier, which requires a `StackMapTable` attribute on every method with branches: for each jump target, an explicit description of the types on the operand stack and in the locals. Computing those correctly is a data-flow analysis with a type lattice including `uninitialized` and merge rules over class hierarchies — a real project in itself, and a dreadful thing to debug while also debugging your code generator. Class files of version 49 and below use the older *inference* verifier, which computes the same information itself and needs no attribute. So: emit `49.0` and the whole problem disappears. Modern JVMs still load version 45–49 class files. Do not plan to use `-noverify` / `-Xverify:none` instead: it is obsolete and ignored on current JDKs. If you later want modern class files, switch from Jasmin to ASM with `ClassWriter.COMPUTE_FRAMES` and let the library do it. (That switch is itself a good week of work and teaches you what the attribute contains.)

THREE MORE THINGS THE VERIFIER WILL CATCH, PAINFULLY

.limit stack / .limit locals must be at least the true maxima. Compute the stack limit by abstract interpretation over your CFG: the stack depth at a block's entry must be the same on every incoming edge — assert that, because a mismatch is also a verify error and your own assertion has a much better message than `VerifyError: Inconsistent stack height`.

Category-2 values. `long` and `double` occupy **two** local slots and two stack words. A slot allocator that assumes one slot per variable produces code that verifies for a while and then does not. Write the test with a `real` local between two `integer` locals.

Stack discipline across jumps. Your local `goto` is only safe between statement boundaries, where the operand stack is empty. Assert “stack is empty at every statement boundary” in the emitter; it makes an entire class of bug impossible.

Boolean expressions: the single most useful idea in Appel

How do you compile `if (a < b) and (c < d) then S`? Naively: compute `a < b` as a 0/1 value on the stack, compute `c < d`, `land`, then `ifeq` past the body. That works and is bad code — the JVM has no comparison instruction that produces a boolean, so each comparison becomes a branch, two loads and a jump. Appel §7.2's answer is to give the translator three kinds of result:

- **Ex** — an expression producing a value;
- **Nx** — a statement producing nothing;
- **Cx** — a *conditional*: something that, given a true-label and a false-label, jumps to one of them.

with conversions between them. A comparison is naturally a **Cx**; `and` of two **Cx**'s is a **Cx** that threads the labels; and only when a boolean genuinely needs to be a value (stored in a variable, passed as an argument) do you convert **Cx** → **Ex** by materialising 0 or 1 around a branch. The generated code for the example above becomes two compare-and-branch instructions and no arithmetic at all. Implement this; it is perhaps 100 lines and it is the difference between a toy code generator and a real one.

A SEMANTIC QUESTION YOU MUST ANSWER FIRST

The **Cx** translation naturally **short-circuits**. ISO Pascal does *not* require short-circuit evaluation of `and` / `or`, and does not require full evaluation either — it leaves the order unspecified, which means `(p <> nil) and (p^.x = 1)` is not portable Pascal even though every practical compiler makes it work. Turbo Pascal has a switch (`$B`). So: which do you implement, do you offer both, and if you offer both, does the choice live in the front end (different HIR) or the back end (different code for the same HIR)? Whichever you pick, note that a program's *meaning* now depends on it, so it belongs in `DEVIATIONS.md` and in a test.

A dozen mapping decisions

PASCAL	JVM REPRESENTATION	THE CATCH
<code>integer</code> , <code>subranges</code> , <code>enums</code> , <code>boolean</code> , <code>char</code>	<code>int</code> on the stack (<code>Z</code> , <code>C</code> , <code>B</code> , <code>S</code> in descriptors)	All ordinals are <code>int</code> in the JVM's operand stack. Keep the Pascal type in MIR; the back end knows the JVM does not distinguish.
<code>real</code>	<code>double</code>	Two slots. Also: Pascal's <code>trunc</code> / <code>round</code> vs <code>d2i</code> semantics differ at the boundaries.

<code>array[lo..hi] of T</code>	Java array of length <code>hi-lo+1</code> ; index as <code>i - lo</code>	The subtraction can defeat HotSpot's bounds-check elimination. But see the note below: it can also <i>give</i> you bounds checking for free.
Multi-dimensional arrays	Nested Java arrays, or one flat array with computed index	Nested is simpler and matches <code>a[i][j]</code> ; flat is faster and is what you need if you ever want a row to be a value rather than a reference. Pascal array assignment copies — so nested arrays need a deep copy on assignment. Decide, and test <code>a := b</code> for a 2-D array.
<code>record</code>	A generated class with public fields	Assignment copies in Pascal. You need a generated <code>clone</code> -like copy. Nested records copy deeply.
<code>set of T</code>	<code>long</code> for ≤ 64 elements, <code>long[]</code> otherwise	<code>set of char</code> is 256 bits = <code>long[4]</code> . Implement <code>+</code> <code>-</code> <code>*</code> <code><=</code> <code>in</code> as bit operations; this is one of the few places you can dramatically beat a naive implementation.
<code>string</code> / packed array of <code>char</code>	<code>char[]</code> (fixed length) or <code>String</code>	ISO strings are fixed-length and blank-padded; <code>String</code> is not. Using <code>String</code> is tempting and subtly wrong for a standard-conforming compiler.
<code>^T</code> , <code>new</code> , <code>dispose</code>	Object reference, <code>new</code> , no-op	<code>nil</code> is <code>null</code> ; a <code>nil</code> dereference is a <code>NullPointerException</code> you must catch and re-report as a Pascal runtime error with a line number.
<code>var</code> parameter	A one-element array, or a generated <code>IntRef</code> -style cell, or (array, index) pair	The argument may be <code>a[i]</code> : evaluate <code>i</code> exactly once, then alias the slot. Copy-in/copy-out is <i>not</i> equivalent when aliasing is observable — construct the program that distinguishes them (Task 11.3).
<code>case</code>	<code>tableswitch</code> or <code>lookupswitch</code>	Choose by label density; the usual heuristic is table if $(\max - \min + 1) \leq 2 \times \text{count}$ or so. Measure once and write the number down.
Local <code>goto</code>	<code>goto</code>	Safe only where the operand stack is empty.
Non-local <code>goto</code>	Throw a dedicated exception caught in the target frame	The frame must be identified dynamically (which activation?), which is why this is in the “Later” bucket.
text files, <code>read</code> / <code>write</code>	Runtime library over <code>System.in/out</code>	Pascal's <code>read</code> tokenising and <code>eoIn</code> / <code>eof</code> semantics are fussier than they look. See M9.

A TRICK WORTH CONSIDERING, AND ARGUING ABOUT

If you emit `i - lo` as the index and let the JVM's own array bounds check fire, then an out-of-range Pascal index becomes an `ArrayIndexOutOfBoundsException`, which you catch at the procedure boundary and re-report. You get bounds checking for free, with zero emitted instructions, optimised by C2. The costs: the exception message is in terms of the shifted index, you cannot distinguish “index below `lo`” from “above `hi`” without extra work, and it does not generalise to your stage-2 VM. Which matters more to you? Whichever you choose, your MIR should still contain the explicit `check.index` instruction — the back end decides how (or whether) to materialise it. That is exactly the division of labour the IR exists to provide.

TASKS

11.1 (1) Emit `.line` directives from MIR source positions, so a Pascal runtime error produces a JVM stack trace naming `hello.pas:17`. Half a day, and it transforms debugging for the rest of the project.

11.2 (2) Compute `.limit stack` two independent ways — tracked during emission, and by a separate abstract interpretation over the finished code — and assert they agree on every method in the corpus. Self-checking invariants like this are how compiler engineers stay sane.

11.3 (1) Write the Pascal program that distinguishes call-by-reference from copy-in/copy-out (pass the same variable as two `var` parameters, or pass a global that the callee also names). Then decide your `var`-parameter implementation with that program as the test.

11.4 (2) Implement sets as bitmasks and benchmark `c in ['a'..'z']` against a naive comparison chain. Then look at what HotSpot does to both.

11.5 (2) Run every generated class through ASM's `CheckClassAdapter` in your test suite, as a verifier you control, with better messages than the JVM's.

Reading

Appel ch. 7 §7.2 (Ex/Nx/Cx) — read this twice; ch. 8 (basic blocks and traces — block ordering to minimise jumps); ch. 9 (instruction selection: maximal munch and dynamic programming) which you can skim for stage 1 and will need for stage 2; ch. 12 for how the pieces join.

Dragon §6.6 (control-flow translation of boolean expressions — the same idea as Cx, presented as inherited attributes), §6.7 (backpatching — the one-pass alternative), §8.6 (a simple code generator), §8.10 (Sethi–Ullman optimal expression code — note that it is about *register* machines and is therefore irrelevant to your JVM back end and highly relevant to your stage-2 register VM: a nice illustration of how target shape changes which theory applies).

Wirth ch. 9–11 — expressions, assignments, conditionals and boolean expressions on a RISC target. Compare his conditional-jump handling directly with Appel's Cx and note they are the same idea discovered twice.

Pierce ch. 19 (Featherweight Java) — useful background when you decide how records and frames become classes.

Outside the books: the JVM Specification ch. 4 (class file format, and §4.10 on verification) and ch. 6 (instruction set). Read §4.10.1 vs §4.10.2 to understand exactly what you are avoiding by emitting version 49.

Testing, conformance and measurement

A compiler is a program whose bugs are silent: it produces output that looks fine and is wrong. The testing strategy is therefore part of the architecture, not an afterthought.

Six kinds of test, in increasing order of value per hour

1. **Unit tests** on the lexer and on small utilities. Cheap, necessary, low yield.
2. **Golden files.** `--emit=tokens|ast|mir` output committed next to each source file. A diff shows you exactly what a change did. The single best return on effort in the whole list, which is why Part 2 insists on a printable IR.
3. **Diagnostic fixtures.** Every reject program asserts the exact set of error codes and positions. Asserting “no extra diagnostics” is what keeps cascades under control.
4. **Execution tests.** Run the compiled program, compare stdout byte for byte with an expected file.
5. **Differential testing.** You have three executors (MIR interpreter, JVM back end, later your VM) and the outside world has Free Pascal (`fpc`) and GNU Pascal. Any disagreement is a bug in exactly one of them, and finding out which is always instructive.
6. **Metamorphic testing.** `-O0` and `-O2` must produce identical output. No expected values needed, so you can point it at *any* Pascal program, including ones you did not write. This is how you find optimiser bugs at scale.

TASKS

12.1 (2) Fuzz the front end: feed it random bytes, random token sequences, and mutated valid programs. The invariant is not “it compiles” but “it never throws an uncaught exception and never hangs”. Jazzer or a hand-rolled mutator both work.

12.2 (3) Write a random *program* generator for a subset of Pascal that is guaranteed to terminate and whose expected output you can compute (e.g. arithmetic over bounded integers, loops with fixed trip counts). This is a Csmith-style oracle and it will find optimiser bugs you would never write a test for. Constraining generation so that programs are both interesting and well-defined is the hard, interesting part.

12.3 (1) Set up the benchmark harness before M11: five programs (a sieve, matrix multiply, n-body, a recursive one such as Ackermann or binary trees, and one set-heavy or string-heavy program), each run with proper JVM warm-up, reporting steady-state and first-run times separately. **Both numbers matter and they will tell different stories** — which is itself the introduction to everything in Part 13.

12.4 (2) Find a copy of the ISO 7185 conformance test suite (it circulates freely) and run it. Record your score. Watching that number go up is the most motivating progress metric this project offers.

QUESTIONS

How do you test that an *error message* is good, as opposed to merely present? — What is the smallest program that distinguishes your compiler from a conforming one, and do you have it in the corpus? — When the MIR interpreter and the JVM back end disagree, what is your procedure for finding out which is wrong — and can you automate the bisection (over passes, over procedures, over statements)?

Stage 2: your own bytecode and an adaptive VM

This is a second project of comparable size to the first. The point of the architecture in Part 2 is that it starts from a working, tested MIR rather than from nothing.

What “dynamic adaptive” actually buys a statically typed language

Be precise about this early, because it determines your whole design. Adaptive optimisation earns its keep in dynamic languages mostly by *discovering types* — inline caches, type feedback, speculative monomorphisation. Pascal hands you all the types up front. So what is left?

- **Profile-directed layout and inlining:** which call sites are hot, which branches are taken, which loops are worth compiling at all. Available only at runtime.
- **Value and range speculation:** this array is always length 100; this divisor is never zero; this subrange variable is always within bounds. These let you delete checks that static range analysis could not prove — with a guard and a deoptimisation path when the speculation fails.
- **Escape analysis and scalar replacement** of your frame objects and `var`-parameter cells, using runtime knowledge of which calls actually happened.
- **Tiering:** interpret cold code (fast to start, small memory), compile hot code (slow to produce, fast to run). Almost all of the practical benefit for a compiler like yours.
- **On-stack replacement:** entering optimised code in the middle of a long-running loop. Without it, a program with one hot loop in `main` never benefits at all.

Design decisions, in the order you must take them

DECISION	OPTIONS, AND THE CONSIDERATION THAT USUALLY SETTLES IT
Stack or register ISA	Register. Your MIR is already three-address and in SSA form, so lowering is near-trivial; register machines execute far fewer instructions per program, which matters enormously for an interpreter where dispatch dominates; and every optimisation you already wrote applies directly. A stack ISA is easier to verify and more compact — that is why the JVM chose it — but you already have a stack back end. Read Shi et al., “Virtual Machine Showdown: Stack Versus Registers”, and the Lua 5.0 implementation paper before deciding against this.
Encoding	Fixed 32-bit words (simple decoding, wasteful) or variable-length bytes (compact, branchy decoder). Start fixed; you can change it later precisely because the assembler and disassembler are separate programs.
Implementation language	A serious fork in the road. Writing the VM in Kotlin keeps one toolchain and lets you reuse the compiler's data structures — but you inherit the JVM's constraints exactly where they hurt an interpreter most: no computed <code>goto</code> , no control over memory layout, no tail calls, and you would be running a JIT on top of a JIT. Writing it in Rust or C gives you threaded dispatch, real memory layout control, and honest measurements. Recommendation: prototype the interpreter in Kotlin to validate the ISA (weeks), then rewrite in Rust/C if you want real performance work (months). Be honest about which project you are doing.
Memory management	You now own it. Appel ch. 13 is a complete short course: mark-sweep, reference counting, copying collection, generational. A semispace copying collector is about 300 lines and is the right first one. It requires precise root finding , which requires stack maps at safepoints — which is the same machinery as deoptimisation metadata.

Deoptimisation	The hardest single piece. To abandon optimised code mid-execution you must reconstruct the interpreter's state (locals, stack, program counter) from the optimised frame. That means every speculation point carries a map from interpreter state to optimised locations — and that inlining, constant folding and register allocation must all maintain it. This is why Part 10 told you to attach live-variable maps to trapping instructions from day one. Read Hölzle, Chambers and Ungar, “Debugging Optimized Code with Dynamic Deoptimization” (1992), before you design the IR for tier 2.
-----------------------	--

A STAGED PLAN FOR STAGE 2

1. **Specify the ISA** as a document, with an assembler and disassembler, before writing the VM.
2. **A verifier for your own bytecode.** You will be generating it; you will generate it wrongly; a verifier turns undebuggable crashes into error messages. (Now you understand why the JVM has one.)
3. **A plain interpreter** that runs the whole corpus, checked against the MIR interpreter and the JVM back end. Measure it. It will be perhaps 30–100× slower than the JVM. That is the baseline the rest of the work is measured against.
4. **Profiling counters** on back-edges and call sites; a tiering threshold; a report of what got hot.
5. **A baseline compiler** for hot methods — template-per-instruction, no optimisation, just removing dispatch overhead. Expect a large win for little cleverness. This is where most of your speed comes from.
6. **OSR**, so loops benefit.
7. **An optimising tier** reusing your MIR passes, plus speculation and deoptimisation.
8. **GC**, once allocation matters.

QUESTIONS WORTH ANSWERING IN WRITING BEFORE YOU START

What measurable goal makes stage 2 a success? (“Faster than the interpreter” is trivially achievable; “faster than the JVM back end” is very hard, because you are competing with two decades of HotSpot. “I understand how HotSpot works because I built a small one” is a perfectly good goal — but say so, or you will judge yourself by the wrong metric.) — Method-based JIT (HotSpot, V8) or tracing JIT (PyPy, LuaJIT)? Tracing is dramatically simpler to get working for loop-heavy code and is a much more unusual thing to have built. — What in your MIR would you change today, knowing stage 2 is coming? Change it now, while there is only one back end to update.

Reading

Appel ch. 9 (instruction selection — now it matters: maximal munch and dynamic-programming tiling for your register ISA), ch. 10–11 (liveness, graph-colouring register allocation — the chapters you skipped for the JVM), **ch. 13 (garbage collection)**, ch. 20–21 (scheduling, memory hierarchy) for context on what your baseline compiler is leaving on the table.

Dragon ch. 8 (code generation for real machines), ch. 10 (instruction-level parallelism) and ch. 12 (interprocedural analysis) become relevant here for the first time.

Wirth ch. 9 — his RISC machine is a fine model for a first bytecode ISA: simple, regular, and small enough to hold in your head.

Outside the books: Ertl & Gregg on efficient interpreters; Ierusalimsky et al. on Lua 5.0's register VM; Hölzle et al. on deoptimisation and on polymorphic inline caches; the Maxine, Jikes RVM and V8/TurboFan literature for how tiering is organised in practice.

Decision log template

Keep one file per significant decision in `docs/decisions/`. Five minutes each; they are what make the project comprehensible to you in six months, and they are also the record of what you learned.

```
# ADR-007: var parameters are passed as reference cells

## Status
Accepted, 2026-04-11

## Context
The JVM has no way to pass a reference to a local variable. Pascal's var
parameters require call-by-reference semantics that are observable when
the same variable is passed twice, or when the callee also reads a global
that the caller passed.

## Options considered
1. Copy-in/copy-out           - fastest; wrong under aliasing
2. One-element Java array     - one allocation per call; simple
3. Generated Ref classes      - one allocation; typed; more generated code
4. (array, index) pair        - no allocation for array elements; two params

## Decision
(3), with escape analysis so a var parameter that the callee only reads is
passed by value. Task 11.3's program is the regression test.

## Consequences
- MIR needs an explicit Cell type; the DVM back end must implement it too.
- Scalar replacement in the optimiser can undo the boxing; see Part 10.
- DEVIATIONS.md: none - this is conforming behaviour.
```

Decisions worth an entry, at minimum: dialect and feature triage; short-circuit or not; overflow policy; string representation; array lower-bound handling; `var`-parameter representation; nested-procedure strategy; class-file version; SSA construction algorithm; pass order; stage-2 ISA shape.

Suggested repository layout

Concrete, for your existing Kotlin/Gradle setup. The module boundaries are the architecture; make Gradle enforce them.

```
settings.gradle.kts
    include(":util", ":frontend", ":mir", ":backend-jasmin", ":runtime", ":driver", ":conformance")
    // stage 2 later: ":backend-dvm", ":dvm"

util/           spans, diagnostics, source map, interning, CLI plumbing
frontend/       lexer, parser, AST, symbols, types, sema, HIR, HIR->MIR lowering
                depends on :util, :mir           -- NOT on any backend
mir/           MIR data types, printer, parser, verifier, interpreter, passes
                depends on :util           -- knows no target
backend-jasmin/ MIR -> .j text, slot allocation, stack limits, class emission
                depends on :util, :mir
runtime/       pascal.rt.* : Io, Sets, Strings, RangeError, Frames
                plain Kotlin/Java, shipped with compiled programs
driver/        CLI, --emit=, pass pipeline, backend selection
                depends on everything
conformance/   the corpus + harnesses: golden, diagnostics, execution,
                differential, metamorphic, benchmarks

docs/
    roadmap.(html|pdf)    this document
    DEVIATIONS.md         where you differ from ISO 7185, and why
    decisions/ADR-*.md    one file per decision (Appendix A)
    grammar.ebnf          the single source of truth for the syntax
    mir.md                the MIR instruction reference

testSources/    (rename from testSources)
    accept/       must compile and run, with expected stdout
    reject/       must fail, with expected diagnostics
    bench/        M10 benchmark programs
```

TWO BUILD DETAILS FOR YOUR SETUP

Your Gradle build targets JVM toolchain 18 — that is the JVM your *compiler* runs on and is unrelated to the class-file version your compiler *emits* (49.0, per Part 11). Keep the two straight in your head and in your build file comments; conflating them is a genuinely confusing afternoon.

Second: add the `jasmin` assembler invocation as a Gradle task or shell out to it from the driver, but keep `.j` generation and assembly as separate steps — you want the `.j` text on disk for golden tests and for reading with your own eyes.

THE FIRST TWO WEEKS, CONCRETELY

1. Rename `testSources/`, split it into `accept/` and `reject/`, and write down why `test0.pas` is in whichever bucket you chose (Part 1).
2. Create the modules above, empty, with the dependency rules in place. Verify `:frontend` compiles with `:backend-jasmin` removed from `settings.gradle.kts`.
3. Commit `grammar.ebnf` — you already have it on paper; making it a file the build can read is what enables Task 6.1.
4. Spans and the diagnostics engine (Part 4), with the caret renderer.
5. The M1 vertical slice: five tokens, one statement, one MIR instruction, one method emitted, one `42` printed by a real JVM. Nothing general, everything connected.

Cross-reference matrix

Where to look when you are stuck on a specific thing. Chapter numbers: Wirth 1996; Dragon 2nd ed.; Appel *Modern Compiler Implementation in Java* 2nd ed.; Pierce *TAPL*.

TOPIC	WIRTH	DRAGON	APPEL	PIERCE
Whole-compiler overview	1–2, 6–7	ch. 1–2	ch. 1, 12	—
Regular languages, scanning	3	ch. 3	ch. 2	—
LL(1), FIRST/FOLLOW	4, 7	§4.4	§3.2	—
Error recovery	7	§4.1.3–4, §4.4.5	§3.4	—
Abstract syntax	5	§2.8, ch. 5	ch. 4	ch. 3 (inductive defs)
Symbol tables, scopes	8	§2.7, §7.1	ch. 5	ch. 6 (nameless terms)
Type checking	8	§6.3–6.5	ch. 5	ch. 8–11, 13
Subtyping / coercion	—	§6.5	—	ch. 15–17
References, pointers, state	13–14	§7.4	—	ch. 13
Runtime errors / exceptions	—	§7.4	—	ch. 14
Activation records, static link	12	§7.1–7.3	ch. 6	—
Closures, lambda lifting	—	—	ch. 15	ch. 5–7
IR design	—	§6.1–6.2	ch. 7	—
Boolean expressions, jumps	11	§6.6–6.7	§7.2 (Cx)	—
Basic blocks, CFG, traces	—	§8.4–8.5	ch. 8	—
Instruction selection	9–10	§8.6–8.9	ch. 9	—
Expression code, Sethi–Ullman	10	§8.10	§9.1	—
Liveness, register allocation	—	§8.8	ch. 10–11	—
Data-flow analysis	—	§9.2–9.3	ch. 17	—
SSA	—	§6.2.4, §9.5	ch. 19	—
Loop optimisation	16	§9.6–9.7	ch. 18	—
Garbage collection	—	§7.5–7.8	ch. 13	—
Objects / class layout	—	§6.?, ch. 7	ch. 14	ch. 18–19
Separate compilation, modules	15	—	—	—

Beyond the four books

Short list. Every item is here because it answers a question the four books leave open for *this specific project*.

SOURCE	WHY, AND WHEN
ISO/IEC 7185:1990, <i>Standard Pascal</i>	Your specification. Freely available. Part 1 onwards — read the sections on assignment compatibility and on <code>for</code> statements before you implement them.
Cooper & Torczon, <i>Engineering a Compiler</i>	The best single complement to your four: more practical than Dragon, more thorough than Appel on IR and optimisation. If you buy one more book, this one.
Muchnick, <i>Advanced Compiler Design and Implementation</i>	The reference for optimiser details — every pass in Part 10, in full, with the corner cases. Reference only; unreadable end to end.
Cytron et al. (1991), <i>Efficiently Computing SSA Form...</i>	The dominance-frontier algorithm. Part 10, Task 10.1.
Braun et al. (2013), <i>Simple and Efficient Construction of SSA Form</i>	SSA without dominance frontiers; what you will actually ship. Part 10.
Wegman & Zadeck (1991), <i>Constant Propagation with Conditional Branches</i>	SCCP. Part 10, Task 10.2.
Cooper, Harvey & Kennedy, <i>A Simple, Fast Dominance Algorithm</i>	Twenty lines, fast in practice. Part 10.
Lindholm et al., <i>The Java Virtual Machine Specification</i> , ch. 4 & 6	Class file format, the verifier (§4.10), the instruction set. Part 11 — the authority when Jasmin and the JVM disagree with you.
Jasmin user guide; the ASM library guide	Part 11. Read ASM's guide even if you stay on Jasmin: it explains the frame computation you are avoiding.
Hölzle, Chambers & Ungar (1992), <i>Debugging Optimized Code with Dynamic Deoptimization</i>	The paper that makes adaptive optimisation possible. Read <i>before</i> designing stage 2's optimised tier.
Hölzle & Ungar, <i>Optimizing Dynamically-Typed OO Languages with Polymorphic Inline Caches</i>	Where “adaptive” came from; useful for understanding what does <i>not</i> apply to Pascal.
Ertl & Gregg, <i>The Structure and Performance of Efficient Interpreters</i>	Dispatch techniques and why your language choice for the VM matters. Part 13.
Ierusalimsky, de Figueiredo & Celes, <i>The Implementation of Lua 5.0</i>	The clearest short description of a register VM in existence. Part 13.
Shi et al., <i>Virtual Machine Showdown: Stack Versus Registers</i>	The evidence for the stack/register decision. Part 13.
Bolz et al., <i>Tracing the Meta-Level</i> (PyPy)	If you choose a tracing JIT over a method JIT. Part 13.
Wirth, <i>Algorithms + Data Structures = Programs</i>	Chapter 5 is a Pascal compiler for PL/0, by Pascal's author. Read it for the pleasure and for the sense of scale.

Risk register and anti-patterns

Every one of these has killed a student compiler. Most of them look like good engineering at the time.

ANTI-PATTERN	WHY IT IS TEMPTING	WHAT TO DO INSTEAD
Perfecting phases in order	It feels orderly, and the lexer is fun.	M1 vertical slice first. Always have something that runs.
A general IR before a second consumer exists	"I'll need it for the VM."	Design MIR for the JVM back end and the interpreter; revisit when the second back end is real. Keeping it printable is the cheap insurance.
Optimising before measuring	Optimisation is the interesting part.	M10 before M11. Write down the zeros as well as the wins.
Silent semantic deviations	Each one is individually trivial.	DEVIATIONS.md, updated the same hour you deviate.
A mutable AST decorated in place	It is the obvious thing, and every textbook does it.	Side tables keyed by node id; immutable nodes. You can then re-run a phase, or run two.
Map<String, Any> symbol tables	Fast to write in a dynamic style.	Sealed Symbol hierarchy; the type checker's bugs become compile errors in <i>your</i> compiler.
Error messages as strings at the throw site	Quickest possible thing.	Codes + structured diagnostics from Part 4. Retrofitting is a day of tedium per phase.
Implicit safety checks	They belong to the store, surely.	Explicit check instructions in MIR, or the optimiser can never remove them.
Starting stage 2 before stage 1 is conformant	The VM is the exciting part.	M8 first. A VM for a language whose semantics you have not pinned down has no specification to be correct against.
Non-determinism in output	HashMap is the default.	LinkedHashMap, sorted diagnostic output. Otherwise golden tests fail on other machines and you will blame the tests.
No --emit flags	Debugger-driven development feels sufficient.	Every phase inspectable from the command line. It is how you will debug for the next two years.

A closing thought on what you are actually building

You have chosen an unusually good project. A Pascal front end is small enough to finish, and Pascal's type system is rich enough (subranges, sets, name equivalence, nested procedures, var parameters) that nothing about it is trivial. The JVM back end forces you to confront a target that is *almost* but not quite a good fit — non-zero array bases, no static links, no

references to locals, a verifier with opinions — and every one of those mismatches is a lesson about what an IR is *for*. And the second stage, where you write the machine as well as the compiler, closes the loop: you will have built both sides of an interface that your four books mostly present as given.

The thing that will determine whether you finish is not any of the technical decisions in this document. It is whether you always have a compiler that works on *some* language, and whether you can see it working. Ship the vertical slice in week two, keep the corpus green, measure before you optimise, and write down every decision as you make it. The rest is chapters.